Automating the DJ Experience

CSE 474 Project Proposal Option: Incorporate ML Model in New Use Case Setting

Alexandra Gelinas
Jaidin Rosario
Sammy Odeh

1 Abstract

In this project, we explore the use of machine learning to automate DJ style transitions between songs. Traditional music models typically focus on generating audio or recommending tracks, but they often fail to capture how songs are actually mixed in practice. Our goal is to build a system that can identify whether two parts of songs will transition smoothly based on their audio features.

We collected data from real DJ sets and extracted features such as tempo, energy, spectral flux, brightness, and MFCCs. Transitions from these sets were labeled as good examples, while negative examples were created using random pairings and modified transitions. We then trained a neural network classifier on feature differences between pairs of song segments to predict transition compatibility.

The model achieved an accuracy of approximately 79.7% and a ROC-AUC score of 0.853, showing that it can effectively distinguish between good and bad transitions. Compared to a simple baseline that only uses tempo, our approach performed significantly better, demonstrating the importance of using multiple audio features.

When tested on real audio, the model was able to identify transitions that sounded smooth in some cases, especially when energy and structure aligned. However, not all high-scoring transitions sounded natural, showing that the model does not fully capture timing and musical context. Overall, this work shows that machine learning can help automate parts of the DJ process, but further improvements are needed for more realistic mixing.

2 Introduction

Current Music AI/ML models do not understand music phrasing and how to “mix” as if it were functioning as a real-life DJ. The goal is for our model to combine advancements such as audio transformation and phrase awareness, creating seamless blends of songs to mimic the feeling of a DJ’s set. We are aiming to combine several components to create a pipeline that enables automated DJ mixing. Firstly, songs from our dataset are analyzed to extract the following features and more, tempo, phrase boundaries, energy progression, key, length, etc. Next, the model is trained to learn the compatibility between songs by learning from transitions from real DJ sets such as Boiler Room Sets or other artists. The parameters for the model include tempo, spectral flux, brightness, energy, and mel-frequency cepstral coefficients. Finally, we will utilize a generative audio module to guide the transition between selected tracks so the output is more continuous than your average cross fade mix. This way, our project can recreate the process of mixing songs comparably to a real-life DJ with years of experience. This saves lots of time and money, which can appeal to users because hiring a DJ can be very expensive and may not be accessible in one’s area. The goal is to create a virtual DJ that understands music phrasing and harmonic compatibilities between songs thus providing clients with set-lists and transitions that match the vibe.

A current challenge in the world of automated DJing is that "acceptable" transitions are very subjective and context dependent. Two songs may be close in key and BPM but can still clash harmonically, while another two songs with different tempos may mix better if their energy and phrase structures match better. As a result of this, an effective virtual DJ system must go a step further than simply matching tempos and keys and instead learn the relationship between songs through their phrasing, and energy curves.

We will use public DJ sets for supervised training of the model to recognize acceptable transitions by basing analyses on variation in bpm, harmonics, and energy curves of the training DJ sets. The model will then generate transitions between 2 or more songs that fit within the acceptance thresholds.

3 Related Works

Recent work in machine learning for music understanding and generation has made a lot of progress in automated audio blends, structure analysis, and playlist generation. We will categorize relevant literature works into four areas: music generation models, music structure analysis, automated DJ systems, and audio representation learning.

3.1 Music Generation Models

Recent generative models have shown significant improvement with producing music that sounds realistic and "flows well". For example, MusicGen [8] lets you control the genre of music to be selected from using text prompts and transformer models. MusicLM [1] also creates music from text, but places a larger focus on generating longer pieces that stay consistent over time. Mustango [16] improves how closely the music matches the client input and provides improved

control over what is generated. By accepting user input, clients are able to create music for any event they choose, from birthday parties to DJ-ing at music festivals.

AudioCraft [5] is a framework that brings models like MusicGen together and simplifies user experience for convenience. Earlier work, like Jukebox [10], showed that it is possible to generate raw audio, but it was not very flexible black. Newer approaches such as AudioLDM [15] and other diffusion-based models [3] produce higher-quality audio with greater variety.

Even with these improvements, most of these models still struggle with musical structure and do not create smooth transitions between songs.

3.2 Music Structure and Phrase Analysis

Understanding musical structure is important for creating smooth and natural transitions. Earlier research has focused on finding things like beats, phrase boundaries, and sections such as verses and choruses. Transformer-based models have been used for beat and downbeat tracking [14], and have proven to work well for rhythmic alignment.

Self-supervised methods [9, 24] can find repeating patterns in music without requiring labeled data. This is helpful in recognizing phrases, which can be defined as specific sequences of human language or melodies. Understanding phrases is key to audio detection, recognition, and transcription. Other work using attention-based models [22] has shown good results in identifying phrase-level structures within audio. Transformers have also been used to model higher-level features such as emotion and energy in music [13].

Although these ideas provide increased understanding of audial data, they are not usually combined with generative models, so most systems still don't use musical structure when creating transitions.

3.3 Automated DJ Systems and Playlist Generation

Automated DJ systems try to copy how human DJs choose and mix songs. Earlier work has looked at recommending transitions between songs and creating playlists using machine learning. For example, reinforcement learning methods [7] learn what users like and try to maintain a stream of recommendations for the user. Other approaches focus on matching similar songs based on their audio features and metadata.

More recent work on automated DJ mixing [26] uses deep learning to create transitions between tracks. Some methods, like neural cross-fading [4], make transitions sound smoother but, they usually overlap songs instead of lining them up based on musical structure. Because of this, these systems do not fully understand phrasing and energy changes, causing their transitions to feel less natural.

3.4 Audio Representation Learning

Accurate and effective audio representation is very important for analyzing and clustering music. The HTS-AT model [6] uses hierarchical token semantic transformers to extract audio embeddings for classification and tagging. CLAP [11] learning joint audio text representations which then uses semantic understanding of music attributes such as mood and genre.

These representations enable downstream tasks such as clustering and similarity modeling. However, some approaches do not use temporal dynamics such as energy curves or phrase alignment, which are essential for DJ style mixing.

3.5 Ethics

The abilities of generative artificial intelligence to create music is rapidly advancing and the ethical concerns are growing along with it. According to Journal for the Interdisciplinary Art and Education, many artists are averse to their music being used by AI models due to plagiarism, creativity, and artistic value [12]. As software engineers, it is important to be reminded of the nuance of using AI in everyday life and considering the question: "Should it be done just because it is possible?"

3.6 Summary

While prior work has made significant progress in music generation, structure analysis, and playlist recommendation, there remains a gap in integrating these components into a unified system for phrase-aware DJ mixing. Our approach aims to bridge this gap by combining structural analysis, compatibility modeling, and generative audio synthesis to produce seamless, DJ-like transitions while maintaining ethical boundaries.

4 Data

For preliminary research and to become familiar with the features of musical data, we collected a network of existing songs based off Spotify's metric of popular songs [2]. With this dataset, we mapped features such as spectral flux, tempo, and energy (Figure 1) against a danceability score. From our results, we decided that energy, flux, and tempo were important features to include as parameters to our machine learning model because data suggested certain ranges were associated with higher danceability scores. We decided to focus on optimizing a model for higher-tempoed and energetic songs that would be adjacent to songs used in Electronic Dance Music (EDM) mixes due to the fact that the volume of data and computational power needed to train a model for multiple genres was not attainable at this time.

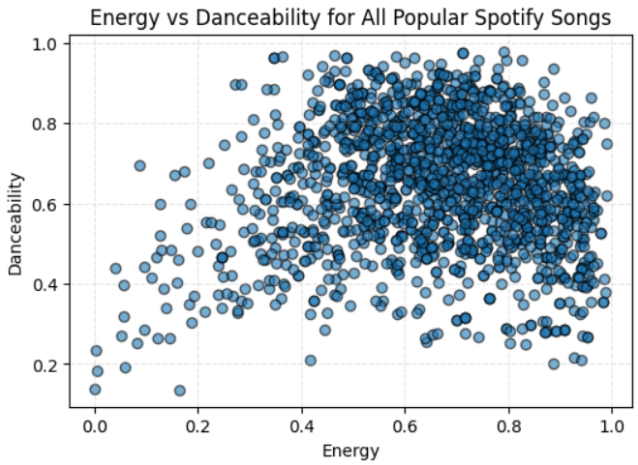


Figure 1: Energy vs Danceability of popular Spotify songs [2]

Data was collected by analyzing popular DJ sets using a custom Python script. Sets were chosen from popularity leaderboards on 1001Tracklists [17]. For example, the performance of a popular artist, Skrillex [23], was chosen due to its number one rank on the date of access. For each set, a text file of song transition times and an mp3 of the performance were used to create data-frames based on features of tracks and transitions. We began the process by manually downloading timestamps and mp3 files, but soon found the process to be tedious and time-consuming. Thus, we built a webscraping loop that extracted timestamps from MixesDB [21] categories and extracted the linked performance on the page to an mp3 file.

Each mp3 was broken into tracks using the song transition times, which allowed us to extract general features using the Librosa [19] library. By comparing features of tracks before and after transitions and the features of each transition phase, we were able to extract features of quality DJ transitions as training data.

Negative data was represented by bad transitions, which means that the audio is a combination of feeling disconnected and songs having low compatability with one another. Low compatability is a concept we investigated by researching the relationship between miscellaneous features against their danceability and energy levels. We had a few human-made transitions in this category, but the majority of negative data was created by skewing feature data from good transitions such that the features of songs were too different to be compatable.

We began the process by manually downloading timestamps and mp3 files, but soon found the process to be tedious and time-consuming. Thus, we built a webscraping loop that extracted timestamps from MixesDB [21] categories and extracted the linked performance on the page to an mp3 file.

4.1 Data Features

Data collected from DJ transitions and songs included:

- change in tempo
- change in flux
- change in energy and loudness
- spectral brightness
- MFCCs

Tempo: Tempo is represented by beats per minute. The speed of a song is important for transitions because significant differences are jarring to listeners and are not considered "danceable." When graphed against danceability, the songs above the 75th percentile ranged between 90 and 140 bpm.

Flux: Flux is essentially the rates in which energy changes within an audio. Our flux measurement comes from Librosa's extttonset_strength feature, which returns a vector representing the spectral flux onset envelope. The average spectral frame is taken from positive changes in Mel bands, which fall under the MFCC category described later. This measurement helps indicate. This measurement helps to capture variation in audio, aiding in recognizing flatness in mixes.

Energy and Loudness: Energy measures the overall intensity of a track and helps capture how forceful or driving it feels, while loudness reflects the perceived volume level of the audio. These features matter in DJ transitions because large mismatches can make a mix feel abrupt or unbalanced, even when tempo and key are compatible. By comparing changes in energy and loudness across songs and transition regions, our model can better identify whether two phrases will blend smoothly or create a noticeable drop or spike in intensity.

Spectral Brightness: Spectral brightness is harmonic content, or tone, of the audio. This feature tries to assign a value to a very subjective and human quality of music. Brightness is often measured using the centroid, which assigns an average for the spectral brightness over the entire audio.

MFCCs: Mel-frequency cepstral coefficients, are compact representations of the short-term power spectrum of a sound, designed to approximate human auditory perception and will aid us in evaluating an acceptance threshold for mixes. The Mel bands represented by these figures allows representation for transient sounds and variations within speech [25]. Each Mel band can be graphed to produce a full spectrogram as shown in Figure 2. In order to experiment with real audio data, we extracted mp3 files from Free Music Archive [18] to use for further analysis and transition generation.

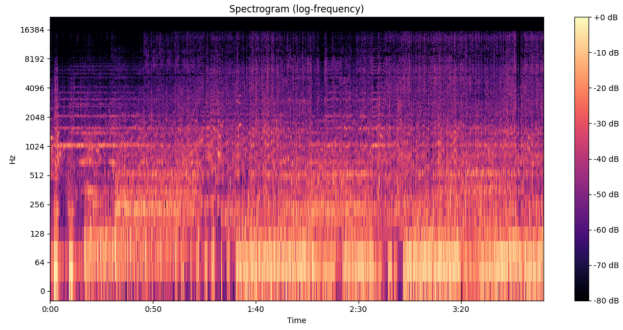


Figure 2: Spectrogram of MFCCs from Garrix 2026 [20]

5 Approach

Instead of building a full end-to-end DJ system, we focused on testing whether our model could correctly identify good and bad transitions between songs. We worked with a subset of songs and DJ sets to build and evaluate our pipeline. This allowed us to focus on the core problem of transition quality without needing a fully polished mixing system.

For each track, we extracted audio features such as tempo, RMS energy, spectral flux, spectral centroid, and MFCCs. These features were chosen because they capture different parts of how a song sounds, including its speed, intensity, and overall tone. Once features were extracted, we compared pairs of song segments by taking the absolute difference between their feature values. This gave us a simple way to measure how similar or different two parts of songs are.

We then used these differences as inputs to a classification model. Real DJ transitions were labeled as positive examples, while negative examples were created using both random pairings and synthetic modifications of real transitions. This setup allowed the model to learn patterns that make transitions feel smooth versus those that feel disconnected.

The main goal of this approach was to test whether these audio features, when combined, could be used to predict realistic and usable transitions between songs. Rather than focusing on perfect audio generation, we prioritized evaluating whether the model could rank and identify transitions that would sound good in practice.

5.1 Audio Feature Extraction

The first stage in our pipeline converts raw audio into a structured table of numerical representations which is then used by our learning model. In one of our custom python files, GetTrainingData.py, we load audio with Librosa and then extract the features described in our Data section, including tempo, RMS energy, spectral centroid, onset-based flux, and MFCC sumamries. This makes it possible for us to move from raw recordings to a suitable format for training.

This step is in place to give the model a compact description of each musical segment so that the later stages can compare segments in a way that has musical significance. Since we are using the same features for our training and inference, the system stays consistent internally. This results in the neural network classifier learning from the same type representation that it later sees when evaluating new transitions.

5.2 Transition Representation

We opted to classify relationships between two tracks/phrases via pairwise feature differences, rather than classifying them independently. For each candidate transition, we utilize the feature vector of the outgoing segment and the feature vector of the incoming segment, then compute their difference. As a result, a pairwise feature-difference vector is produced that summarizes how far apart the two segments are in tempo, energy, brightness, and activity. This is the central idea behind training our model, effectively mapping data vectors to transition quality. In the context of mixing songs, a segment is not "good" nor "bad" when by itself. The important part is whether it has a natural flow from the previous segment. Since we convert each candidate transition into a difference vector, the system learns what changes tend to sound acceptable and which ones feel abrupt, disconnected, or unnatural. This helps our approach simulate real DJ behavior more accurately.

5.3 Classification of Transition Compatibility

After transitions are represented as pairwise difference vectors, the problem is framed as supervised binary classification. Our positive examples come from transitions observed in real DJ sets, while negative examples represent low-compatibility transitions. In the early stages of development, the randomly matched negatives were useful as a baseline, but they made the problem too simplified and overlooked the idea that two random samples could form a "good" transition. As a result, we shifted the method towards data-oriented negative examples, including creating synthetic negatives by altering realistic transitions so that the incoming segment significantly differs from the outgoing segment in tempo, energy, key, or brightness. Our model's ability was no longer limited to detection of extreme mismatches, which resulted in deeper and more meaningful learning.

The heart of our approach is the neural network classifier. In our notebook, transition vectors are standardized using **StandardScaler**, then passed into a multilayer perceptron implemented with **MLPClassifier** from scikit-learn. We utilized sizes of (64,32) for our hidden layers in an effort to give enough capacity to learn nonlinear relationships among the input features. A transition cannot be judged by tempo or energy alone, because several features interact at the same

time, which made musical data better to represent by non-linear behavior. In the case where there is a moderate tempo mismatch, the transition may still work when the energy profile and timbre align. On the other hand, even a small tempo difference can fail if the two phrases feel too different in intensity. The neural network is well suited to determine the correct decision because it can learn from combinations of cues, rather than relying on a single threshold.

The neural network classifier is also what turns our system from a descriptive feature analysis, into a predictive model. Instead of manually declaring what counts as a smooth transition, we let the network learn from the provided examples. In addition to the class label, we also output a probability score, which can be interpreted as a measure of transition compatibility. This score becomes the main signal used to rank our candidate transitions in our pipeline (Figure 4).

5.4 Phrase Level Transition Inference

To better reflect how DJs actually mix, we evaluated transitions at the phrase level. We divided songs into short fixed-length phrases, with the first and last ten seconds removed so that intros, fade-ins, fade-outs, and other low-information regions dont dominate the analysis. In order for our model to focus on sections that had more potential as realistic mixing candidates, we filtered out low-energy phrases.
For each phrase, the same feature family is extracted, and many candidate phrase pairs are formed across different songs. Each pair gets converted into a pairwise feature difference vector in the same way as training. With these vectors, we produce compatibility probabilities for each possible transition. This is where our model becomes useful, because it determines which phrase pairings are most promising.

5.5 Transition Creation and Evaluation

After the highest scoring phrase pairs are identified, we generate transition previews to test whether strong model scores directly correspond to smooth, high quality transitions. Its created by using Librosa's time-stretching tools to tempo align the second audio segment to the first. The transition is synthesized by taking the end of one segment, the beginning of the next, and then combining them through a short crossfade, this way we can listen to the model's recommendations rather than looking at numerical scores.

6 Experiments and Results

We compared a feature based approach with a tempo only approach. The tempo only method selected tracks based on similar BPM and energy, while the chemistry based method used additional features like MFCCs, spectral flux, centroid, and RMS to evaluate compatibility between songs.

Then we analyzed MFCC differences across known DJ transitions to understand patterns in a good transition. we also generated negative samples by randomly pairing tracks, and later improved this by filtering out low energy phrases and restricting phrase selection to the range between 10 seconds and the end minute 10 seconds to avoid silent or faded sections. This is something we want to improve on in the future to allow for more challenging and professional transitions to approach

We extracted phrases from songs, computed feature differences between phrase pairs, and then used these as inputs to a classification model. The model outputs a probability score representing how compatible two phrases are for a transition. To evaluate the system, we ranked phrase pairs based on their predicted scores and then generated audio transitions using tempo alignment and cross fading. This allowed us to actually listen to the transitions and qualitatively assess if the higher scoring pairs actually resulted in smoother mixes. We then also visualized results using plots.

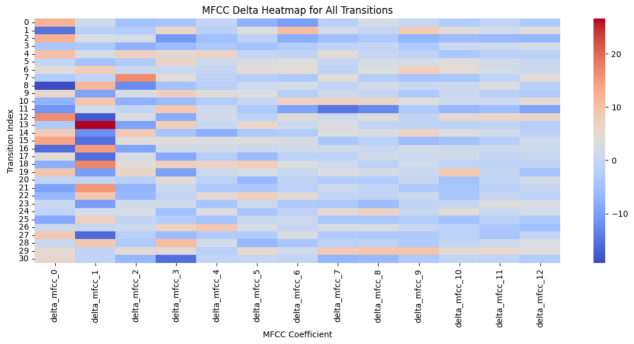


Figure 3: MFCCs of Skrillex [17] transitions

Figure 3 shows MFCC differences across a real DJ transitions. We observed that most values remain small, which would indicate that good transitions preserve similar timbrel characteristics rather than introducing large spectral changes. This supports our approach of using MFCC similarity as a main feature for predicting compatibility.

We also tested some early transition generation to better understand the strengths and limitations of our system. In one successful case, Song A was mixed out into Song B at a convenient phrase boundary where Song A was beginning to reduce in intensity as Song B (the incoming song) was entering with a percussion heavy intro. The Free Music Archive [18] audio clips were used to blend and generate transitions and then classified as "good" or "bad" to test our model. At our current point of development, we face issues with overfitting out model, with the model having a tendency to classify all transitions as "bad" (Figure 4). We suspect this issue comes from a multitude of reasons, one of which is the idea that MFCCs and changes in flux are very subtle measurements, with songs in the same genre having very small differences. This insinuates that the model does not have enough data to compensate for the sensitive nature of

the metric because any given song is likely to be mixed with a song of similar genre. In order to compat this, we aim to tweak our model to include data from the HTS-AT [6] in order to expand upon the metrics denoting a "good" mix.

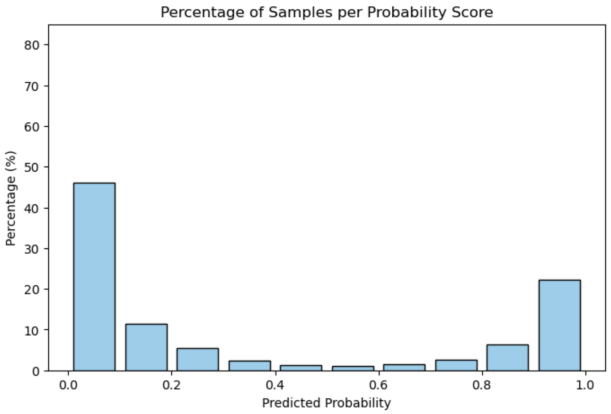


Figure 4: Bad(0.0) vs Good(1.0) Classification

Figure 4 shows the distribution of predicted transition scores from our model. The results are highly skewed currently, with most predictions near 0 and a small number near 1. This suggests that the model is overfitting as it tends to push predictions toward extreme values (0/1) instead of producing a smooth range of scores. This is something we want to change in the future by loosening the decision boundary and improving our negative sampling strategy so the model is forced to learn more subtle differences between transitions rather than making overly confident predictions.

7 Results

Overall, our model showed solid performance in classifying good and bad transitions between songs. It was able to correctly identify most real DJ transitions and assign higher scores to pairs of segments that were more compatible. Compared to a simple baseline that only uses tempo, our model performed significantly better, showing that using multiple audio features helps capture more meaningful relationships between songs.

When we tested the model on real audio, it was able to find some transitions that sounded smooth and natural, especially when the energy and structure of the songs matched well. However, not all high scoring transitions sounded good when played back. This suggests that while the model is learning useful patterns, it still misses some important aspects of how DJs mix music in practice.

```
Train shape: (2542, 9) (2542,)
Test shape: (636, 9) (636,)

Accuracy: 0.7971698113207547
ROC-AUC: 0.853022032356315

Confusion matrix:
[[196 122]
 [  7 311]]

Classification report:
              precision    recall  f1-score   support

           0       0.97      0.62      0.75       318
           1       0.72      0.98      0.83       318

    accuracy                           0.80       636
   macro avg       0.84      0.80      0.79       636
weighted avg       0.84      0.80      0.79       636


Threshold comparison:

Threshold: 0.5
Accuracy: 0.7877358490566038
Precision: 0.7056179775280899
Recall: 0.9874213836477987
F1: 0.8230668414154653

Threshold: 0.6
Accuracy: 0.7877358490566038
Precision: 0.7084282460136674
Recall: 0.9779874213836478
F1: 0.821664464993395

Threshold: 0.65
Accuracy: 0.7971698113207547
Precision: 0.7182448036951501
Recall: 0.9779874213836478
F1: 0.8282290279627164

Threshold: 0.7
Accuracy: 0.7971698113207547
Precision: 0.7192575406032483
Recall: 0.9748427672955975
```

Figure 5: Distribution of model predicted transition scores, showing separation between bad and good transitions

7.1 Quantitative Model Performance

To evaluate the effectiveness of our classification mode, we conducted quanitative analysis using a held test set. The dataset was balanced, consisting of both real DJ transitions and synthetically generated negative examples.

The trained neural network achieved an overall accuracy of 79.7% on the test set. In addition to accuraacy, we evaluated the model using ROC-AUC, which reched a value of 0.853. This shows that our classifier is able to distinguish between camptible and imcompatible transitions across a range of decision thresholds.

To better understand our models behavior, we analyzed the confusion matrix. The model correctly identified the majority of positive transitions, while maintaining low false negative rates. There was a lot of false positives however, which could indicate that some transitions predicted as good may not always produce smooth results.

This further shows that the transition quality is not strictly binary, and many transitions that are technically dissimar may stil sound acceptable based on the style or context of the music.

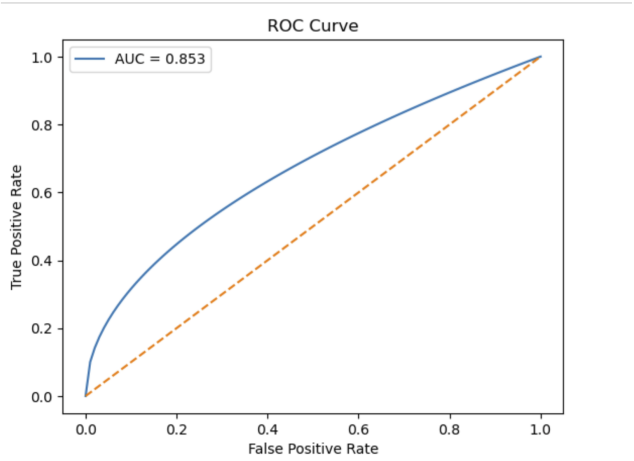


Figure 6: ROC curve (AUC = 0.853)

7.2 Precision Recall Tradeoff

We evaluted the model across multiple probability thresholds to examine the tradeoff between precision and recall. Lower thresholds results in very high recall, meaning that most valid transitions were identified correclt.y This came at the cost of reduced precision though, as more poor transitions were incorrectly labeled as good.

As the threshold increased, precision improved while recall decreased. A threshold of .65 provided the best balance, which had the highest F1 score amongst the tested values. This threshold was selected for downstream evaluation and audio generation.

7.3 Comparison with Baseline Approach

To check if our model is actually learning something useful, we compared it to a simple baseline that only looks at tempo difference. In this baseline, a transition is labeled as "good" if the tempo difference between two tracks is small enough.

This baseline got an accuracy of about 61.6%, which is much lower than our neural network model. This shows that just matching tempo is not enough to decide if two songs mix well.

Tempo is still important for beat matching, but it does not include other things like sound texture, energy level, or how the songs feel together. By using more features, our model can make better decisions about which transitions will sound smoother.

7.4 Distribution of Transition Scores

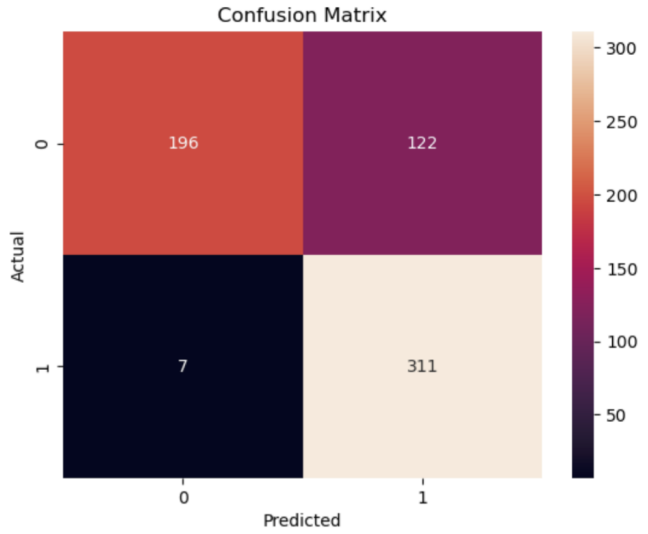


Figure 7: Distribution of model-predicted transition scores, showing separation between low (bad) and high (good) compatibility transitions

We also looked at how the model's prediction scores were spread out. As shown in Figure 4, most of the scores are very close to either 0 or 1, with not many values in between.

This means the model is usually very confident in its predictions. While that can be good, it may also be a sign that the model is overfitting and not learning more subtle differences between transitions.

Ideally, we would want a wider range of scores so we can better tell the difference between average transitions and really strong ones. Right now, the model has a harder time making these smaller distinctions.

This behavior is also reflected in the confusion matrix shown in Figure 6, where the model tends to make confident predictions, leading to strong classification in some cases but also noticeable misclassifications.

This suggests that we may need better negative samples or more training data so the model can learn more detailed patterns.

7.5 Phrase-Level Evaluation and Audio Output

We also tested the model in a more realistic way by creating transitions between short sections (phrases) of songs. For each pair, the model gave a score, and we picked the highest ones to actually generate audio transitions.

To create the transitions, we used tempo matching and crossfading. This let us listen to the results and judge how smooth they sounded. In some cases, the model worked well. For example, transitions sounded good when one song was fading out while the next one started building up.

However, not all high scoring transitions sounded smooth. Some still felt sudden or off, even though the model gave them high scores. This shows that even if two songs have similar features, that does not always mean they will mix well.

This suggests that the model is missing some important parts of music, like exact timing, beat alignment, and key (harmony).

7.6 Key Observations

From these experiments, we noticed a few important patterns in how the model behaves.

- The model is very good at finding valid transitions, mainly because it has high recall. It rarely misses good transitions.
- Using multiple audio features clearly helps. The model performs much better than when using only simple features like tempo.
- The prediction scores are often very extreme (close to 0 or 1), which suggests the model is sometimes too confident in its decisions.
- Even when the model gives a high score, the transition does not always sound smooth when we listen to it.

Overall, these results show that the model is learning useful patterns, but there is still room for improvement, especially when it comes to making transitions sound natural in practice.

References

[1] Andrea Agostinelli et al. 2023. MusicLM: Generating Music From Text. *arXiv preprint arXiv:2301.11325* (2023).
[2] Solomon Ameh. 2026. Spotify Music Dataset. https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset/data. Accessed: 2026-03-28.
[3] Various Authors. 2024. Diffusion Models for Audio Generation: A Survey. *arXiv preprint* (2024).
[4] Rachel Bittner et al. 2023. Neural Crossfading for Seamless Music Transitions. *arXiv preprint* (2023).
[5] Zal'an Borsos et al. 2023. AudioCraft: A Unified Framework for Audio Generation. *arXiv preprint arXiv:2306.05284* (2023).
[6] Ke Chen et al. 2022. HTS-AT: A Hierarchical Token-Semantic Audio Transformer. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2022).
[7] X. Chen et al. 2023. Automatic Playlist Generation Using Reinforcement Learning. *IEEE Transactions on Multimedia* (2023).
[8] Jade Copet et al. 2023. MusicGen: Simple and Controllable Music Generation. *arXiv preprint arXiv:2306.05284* (2023).
[9] Alexandre D'efossez et al. 2023. Self-Supervised Learning for Music Structure Analysis. *arXiv preprint* (2023).
[10] Prafulla Dhariwal et al. 2020. Jukebox: A Generative Model for Music. *arXiv preprint arXiv:2005.00341* (2020).
[11] Benjamin Elizalde et al. 2023. CLAP: Learning Audio-Text Representations. *arXiv preprint arXiv:2211.06687* (2023).
[12] Gvantsa Ghvinjilia. 2026. A review of ethical issues in AI-generated music. 2025, Volume: 6 (01 2026), 367 – 374. doi:10.5281/zenodo.18016622
[13] Q. Huang et al. 2023. A Transformer-Based Approach for Music Emotion Recognition. In *Proceedings of ICASSP*.
[14] Y. Hung et al. 2023. Transformer-Based Beat and Downbeat Tracking. In *Proceedings of ISMIR*.
[15] Zhifeng Kong et al. 2023. AudioLDM: Text-to-Audio Generation with Latent Diffusion Models. In *Proceedings of ICML*.
[16] Jan Melechovsky et al. 2024. Mustango: Toward Controllable Text-to-Music Generation. *arXiv preprint* (2024).
[17] NA. 2026. 1001Tracklists. https://1001.tl. Accessed: 2026-04-21.
[18] NA. 2026. Free Music Archive. https://freemusicarchive.org/. Accessed: 2026-03-31.
[19] NA. 2026. Librosa. https://librosa.org/. Accessed: 2026-03-30.
[20] NA. 2026. Martin Garrix @ SIAM Songkran Music Festival. https://1001.tl/1k0vvwnt. Accessed: 2026-04-15.
[21] NA. 2026. MixesDB. https://www.mixesdb.com. Accessed: 2026-04-05.
[22] Geoffroy Peeters. 2023. Music Structure Analysis Using Attention-Based Models. In *Proceedings of ISMIR*.
[23] QuakePro. 2026. Skrillex @ Mainstage, Ultra Music Festival Miami, United States 2015-03-30. https://1001.tl/533j0kt. Accessed: 2026-03-30.
[24] Colin Raffel et al. 2023. Learning Musical Structure with Self-Supervision. *arXiv preprint* (2023).
[25] Martin A. Trefzer Rinku Sebastian, Rinku Sebastian. 2025. Audio Signal Processing Using Time domain Mel Frequency wavelet coefficient. https://arxiv.org/html/2510.24519v2. (2025). Accessed: 2026-03-26.
[26] Daniel Stoller et al. 2023. Automatic DJ Mixing with Deep Learning. In *Proceedings of ISMIR*.